COMP4141

Theory of Computation

Lecture 11: Resource-Bounded Computation

Outline

Resource-bounded computation

So far we have focused on whether or not we can compute the answer to a problem.

We now focus on the **cost** of computing an answer.

Applications:

- Comparison of algorithms
- Analysis of algorithms (e.g. behaviour, worst-case performance)
- Computation in resource-limited environments

Costs

Two main costs:

- **Time**: How long the computation takes
- **Space**: How much memory the computation uses

Cost as a **function** of parameters associated with the input (usually **input size**)

- Comparing/analysing algorithms that solve "general" problems (i.e. arbitrary instances)

Here we focus on **worst-case** cost analysis

Time Complexity

Definition

The *time complexity* of a deterministic TM that halts on all inputs is the function $f : \mathbb{N} \longrightarrow \mathbb{N}$, where $f(n)$ is the maximum number of steps that M uses on any input of length n.

Time Complexity

Definition

The *time complexity* of a non-deterministic TM that halts on all inputs is the function $f : \mathbb{N} \longrightarrow \mathbb{N}$, where $f(n)$ is the maximum number of steps that M uses on any branch of its computation on any input of length n.

Space Complexity

Definition

The *space complexity* of a deterministic TM that halts on all inputs is the function $f : \mathbb{N} \longrightarrow \mathbb{N}$, where $f(n)$ is the maximum number of tape cells that M scans on any input of length n.

Space Complexity

Definition

The *space complexity* of a non-deterministic TM that halts on all inputs is the function $f : \mathbb{N} \longrightarrow \mathbb{N}$, where $f(n)$ is the maximum number of tape cells that M scans on any branch of its computation on any input of length n.

Intuitively, we expect space to beat time because space can be reused whereas time cannot. (At least I haven't found a way yet.)

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

We know that $\{ 0^i 1^i \mid i \in \mathbb{N} \}$ is a CFL and decidable, e.g. by TM M_1:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found.
2. Repeat as long as there are 0s and 1s on the tape:
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1.
3. If either 0s or 1s are left REJECT, otherwise ACCEPT.

Implementation

How much time does M_1 need, as a function f of the length of the input word w?

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

We know that $\{ 0^i 1^i \mid i \in \mathbb{N} \}$ is a CFL and decidable, e.g. by TM M_1:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found.
2. Repeat as long as there are 0s and 1s on the tape:
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1.
3. If either 0s or 1s are left REJECT, otherwise ACCEPT.

Implementation

How much time does M_1 need, as a function f of the length of the input word w?

w	ϵ	01	$0^2 1^2$	$0^3 1^3$	$0^4 1^4$	$0^5 1^5$
$f(\lvert w \rvert)$	1	12	25	42	63	88

Outline

Asymptotic Behaviour

- The exact cost function is often complicated.
- Running time is sensitive to the specific TM: can get aribtrary (linear) speed-up
- More useful to analyse how the cost **scales**

Asymptotic Notation (Big-O)

Definition

Let $f, g : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$. Say that $f(n) \in O(g(n))$ if there exist $c, n_0 > 0$ such that for all $n \geq n_0$

$$f(n) \leq c \cdot g(n) \ .$$

(Equivalently: $\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$)
g is an (*asymptotic*) *upper bound* for f.

Asymptotic Notation (Big-O)

Definition

Let $f, g : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$. Say that $f(n) = O(g(n))$ if there exist $c, n_0 > 0$ such that for all $n \geq n_0$

$$f(n) \leq c \cdot g(n) \ .$$

(Equivalently: $\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$)
g is an (*asymptotic*) *upper bound* for f.

Bounds of the form n^c for some $c > 0$ are called *polynomial bounds*;
those of the form $2^{(n^\delta)}$ are called *exponential bounds* when $\delta \in \mathbb{R}$ is positive.

Examples

Examples

$5n^3 + 2n^2 + 22n + 6 = O(n^3)$

$\log_a n = O(\log_2 n)$ because $\log_a n = \frac{\log_b n}{\log_b a}$

$2^{\log n} = O(n)$

$O(f(n)) + O(g(n)) = O(f(n) + g(n))$

$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$

Examples

Examples

$5n^3 + 2n^2 + 22n + 6 = O(n^3)$

$\log_a n = O(\log n)$ because $\log_a n = \frac{\log_b n}{\log_b a}$

$2^{\log n} = O(n)$

$O(f(n)) + O(g(n)) = O(f(n) + g(n))$

$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Recall M_1 for deciding $\{ 0^i 1^i \mid i \in \mathbb{N} \}$:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found.
2. Repeat as long as there are 0s and 1s on the tape:
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1.
3. If either 0s or 1s are left reject, otherwise accept.

.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Recall M_1 for deciding $\{ 0^i 1^i \mid i \in \mathbb{N} \}$:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found. $O(n)$
2. Repeat as long as there are 0s and 1s on the tape:
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1.
3. If either 0s or 1s are left reject, otherwise accept.

.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Recall M_1 for deciding $\{ 0^i 1^i \mid i \in \mathbb{N} \}$:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found. $O(n)$
2. Repeat as long as there are 0s and 1s on the tape:
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1. $O(n)$
3. If either 0s or 1s are left reject, otherwise accept.

.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Recall M_1 for deciding $\{ 0^i 1^i \mid i \in \mathbb{N} \}$:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found. $O(n)$
2. Repeat as long as there are 0s and 1s on the tape: $O(n)$
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1. $O(n)$
3. If either 0s or 1s are left reject, otherwise accept.

.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Recall M_1 for deciding $\{ 0^i 1^i \mid i \in \mathbb{N} \}$:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found. $O(n)$
2. Repeat as long as there are 0s and 1s on the tape: $O(n)$
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1. $O(n)$
3. If either 0s or 1s are left reject, otherwise accept. $O(n)$

.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Recall M_1 for deciding $\{ 0^i 1^i \mid i \in \mathbb{N} \}$:

On input w:

1. Scan w and reject if anything not in $\{\sqcup, 0, 1\}$ or 10 is found. $O(n)$
2. Repeat as long as there are 0s and 1s on the tape: $O(n)$
   - Scan across and replace with blanks both the leftmost 0 and the rightmost 1. $O(n)$
3. If either 0s or 1s are left reject, otherwise accept. $O(n)$

$f(n)$ from M_1 is $O(n^2)$.

Big-O Quick Guide

Observation

- For all $k, \epsilon > 0$:

$$O((\log n)^k) \subsetneq O(n^\epsilon) \quad \text{and} \quad O(n^k) \subsetneq O((1+\epsilon)^n).$$

- All logarithms have the same order, irrespective of base:

$$O(\log_2 n) = O(\log_3 n) = \ldots = O(\log_{10} n) = \ldots$$

- Exponentials to different bases have different orders:

$$O(r^n) \subsetneq O(s^n) \subsetneq O(t^n) \ldots \quad \text{for} \quad r < s < t \ldots$$

- Similarly for polynomials

$$O(n^k) \subsetneq O(n^l) \subsetneq O(n^m) \ldots \quad \text{for} \quad k < l < m \ldots$$

Small-o Notation

Definition

Let $f, g : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$. Say that $f(n) = o(g(n))$ if

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0 \ .$$

that is, for any $c > 0$ there exist $n_0 > 0$ such that $f(n) < c \cdot g(n)$, for all $n \geq n_0$.

Small-o Notation

Definition

Let $f, g : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$. Say that $f(n) = o(g(n))$ if

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0 \ .$$

that is, for any $c > 0$ there exist $n_0 > 0$ such that $f(n) < c \cdot g(n)$, for all $n \geq n_0$.

Observe that

1. $f = O(f)$ but $f \neq o(f)$.
2. $f = o(g) \Rightarrow f = O(g)$ but in general $f = o(g) \not\Leftarrow f = O(g)$

Outline

Complexity Classes

A **complexity class** is a class of problems whose solutions' costs have similar properties.

Typically:

- Upper-bounded classes: For each problem, there exists an algorithm whose (worst-case) cost is bounded above
- Lower-bounded classes: For each problem, every algorithm's (worst-case) cost is bounded below

Example

- **RE** is an upper-bounded complexity class
- **Undec**, the class of undecidable problems, is a lower-bounded complexity class

Complexity Classes

A **complexity class** is a class of problems whose solutions' costs have similar properties.

Typically:

- Upper-bounded classes: For each problem, there exists an algorithm whose (worst-case) cost is bounded above
- Lower-bounded classes: For each problem, every algorithm's (worst-case) cost is bounded below

NB

Proving membership of upper-bounded classes is (generally) straightforward. Proving membership of lower-bounded classes is difficult (use reductions!)

Time Complexity Classes

Definition

Let $t : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$. Define the *time complexity class*, $\mathbf{TIME}(t(n))$ to be the collection of all languages that are decidable by an $O(t(n))$ time TM.

Example

Recall M_1 for deciding $A = \{ 0^i 1^i \mid i \in \mathbb{N} \}$. Our analysis of M_1's running time showed that $A \in \mathbf{TIME}(n^2)$.

Time Complexity Classes

Definition

Let $t : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$. Define the *time complexity class*, $\mathbf{TIME}(t(n))$ to be the collection of all languages that are decidable by an $O(t(n))$ time TM.

Example

Recall M_1 for deciding $A = \{ 0^i 1^i \mid i \in \mathbb{N} \}$. Our analysis of M_1's running time showed that $A \in \mathbf{TIME}(n^2)$.

Could we do better for A?

Example: $\left\{ 0^i 1^i \mid i \in \mathbb{N} \right\}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring.
2. Repeat as long as both `0`s and `1`s are on the tape:
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape.
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s.
3. If neither `0`s nor `1`s are left accept, otherwise reject.

Implementation

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring. $O(n)$
2. Repeat as long as both `0`s and `1`s are on the tape:
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape.
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s.
3. If neither `0`s nor `1`s are left accept, otherwise reject.

Implementation

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring. $O(n)$
2. Repeat as long as both `0`s and `1`s are on the tape:
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape. $O(n)$
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s.
3. If neither `0`s nor `1`s are left accept, otherwise reject.

Implementation

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring. $O(n)$
2. Repeat as long as both `0`s and `1`s are on the tape:
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape. $O(n)$
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s. $O(n)$
3. If neither `0`s nor `1`s are left accept, otherwise reject.

Implementation

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring. $O(n)$
2. Repeat as long as both `0`s and `1`s are on the tape:
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape. $O(n)$
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s. $O(n)$
3. If neither `0`s nor `1`s are left accept, otherwise reject. $O(n)$

Implementation

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring. $O(n)$
2. Repeat as long as both `0`s and `1`s are on the tape: $O(\log n)$
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape. $O(n)$
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s. $O(n)$
3. If neither `0`s nor `1`s are left accept, otherwise reject. $O(n)$

Implementation

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n^2)$?

Consider M_2

On input w:

1. Scan w left to right and reject if `10` occurs as a substring. $O(n)$
2. Repeat as long as both `0`s and `1`s are on the tape: $O(\log n)$
   - i. Scan from right to left and reject if there's an odd number of non-`X`s on the tape. $O(n)$
   - ii. Scan from left to right and replace every other `0` by an `X`, beginning with the first `0`. Do the same for the `1`s. $O(n)$
3. If neither `0`s nor `1`s are left accept, otherwise reject. $O(n)$

Implementation

So $L(M_2) = A \in \mathbf{TIME}(n \log n)$.

Comparison of M_1 and M_2

Example

Compare the running times (step numbers) of M_1 and M_2.

w	ϵ	01	$0^2 1^2$	$0^3 1^3$	$0^4 1^4$	$0^5 1^5$
$f_{M_1}(\lvert w\rvert)$	1	12	25	42	63	88
$f_{M_2}(\lvert w\rvert)$	1	15	35	49	81	99

So M_1 beats M_2 at least for short inputs. For $0^9 1^9$ this is no longer the case.

Comparison of M_1 and M_2

Example

Compare the running times (step numbers) of M_1 and M_2.

w	ϵ	01	$0^2 1^2$	$0^3 1^3$	$0^4 1^4$	$0^5 1^5$
$f_{M_1}(\lvert w\rvert)$	1	12	25	42	63	88
$f_{M_2}(\lvert w\rvert)$	1	15	35	49	81	99

So M_1 beats M_2 at least for short inputs. For $0^9 1^9$ this is no longer the case.

Could we do still better for A?

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n \log n)$?

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n \log n)$?

We won't prove this here, but the answer is no, not with a deterministic single-tape TM.

Consider the two-tape TM M_3:

On input w:

1. Scan from left to right and copy `0`s onto the second tape until the first `1` occurs.
2. Keep scanning left to right across the `1`s while scanning right to left on the second tape.
3. Accept if both heads encounter their first blank at the same time, otherwise reject.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n \log n)$?

We won't prove this here, but the answer is no, not with a deterministic single-tape TM.

Consider the two-tape TM M_3:

On input w:

1. Scan from left to right and copy `0`s onto the second tape until the first `1` occurs. $O(n)$
2. Keep scanning left to right across the `1`s while scanning right to left on the second tape.
3. Accept if both heads encounter their first blank at the same time, otherwise reject.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n \log n)$?

We won't prove this here, but the answer is no, not with a deterministic single-tape TM.

Consider the two-tape TM M_3:

On input w:

1. Scan from left to right and copy `0`s onto the second tape until the first `1` occurs. $O(n)$
2. Keep scanning left to right across the `1`s while scanning right to left on the second tape. $O(n)$
3. Accept if both heads encounter their first blank at the same time, otherwise reject.

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n \log n)$?

We won't prove this here, but the answer is no, not with a deterministic single-tape TM.

Consider the two-tape TM M_3:

On input w:

1. Scan from left to right and copy `0`s onto the second tape until the first `1` occurs. $O(n)$
2. Keep scanning left to right across the `1`s while scanning right to left on the second tape. $O(n)$
3. Accept if both heads encounter their first blank at the same time, otherwise reject. $O(1)$

Example: $\{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example

Is there a TM that decides A asymptotically more quickly, that is, is $A \in \mathbf{TIME}(t(n))$ for some $t = o(n \log n)$?

We won't prove this here, but the answer is no, not with a deterministic single-tape TM.

Consider the two-tape TM M_3:

On input w:

1. Scan from left to right and copy `0`s onto the second tape until the first `1` occurs. $O(n)$
2. Keep scanning left to right across the `1`s while scanning right to left on the second tape. $O(n)$
3. Accept if both heads encounter their first blank at the same time, otherwise reject. $O(1)$

So $L(M_3) = A \in \mathbf{TIME}_{2-tape}(n)$.

Complexity vs. Computability

In computability theory we proved that the various TM models (deterministic vs non-deterministic, single-tape vs multi-tape) were equally powerful.

In complexity theory the choice of TM models affects the time complexity of languages.

Multi-Tape TM vs. Single-Tape TM

Theorem

Let $t : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$ *be such that* $\forall n \in \mathbb{N}\,(t(n) \geq n)$. *Every* t*-time multi-tape TM has an equivalent* $O((t(n))^2)$ *time single-tape TM.*

Multi-Tape TM vs. Single-Tape TM

Theorem

Let $t : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$ be such that $\forall n \in \mathbb{N}\,(t(n) \geq n)$. Every t-time multi-tape TM has an equivalent $O((t(n))^2)$ time single-tape TM.

Proof.

By analysing the time complexity of the construction given to show that every multi-tape TM M has an equivalent single-tape TM S.

Since for every step of M, the 1-tape simulator S might have to scan all of the tape used so far, the single step number k of M may cost S up to $O(k)$ steps. Hence the running time of S on an input of length n is

$$O\left(\sum_{k=1}^{t(n)} k\right) = O\left(\frac{t(n) \cdot (t(n)+1)}{2}\right) = O\left((t(n))^2\right) \ .$$

□

Non-Deterministic TM vs. (Deterministic) TM

The *running time* of a deciding non-deterministic TM N on an input word w is the maximum number of steps N uses *on any branch* of its computation tree when starting on w.

Theorem

Let $t : \mathbb{N} \longrightarrow \mathbb{R}_{\geq 0}$ be such that $\forall n \in \mathbb{N}\,(t(n) \geq n)$. Every t-time non-deterministic TM has an equivalent $2^{O(t)}$ time single-tape TM.

Remark: $f = 2^{O(t)}$ means $f(n) \leq 2^{c \cdot t(n)}$ for some constant c.

NDTM $\rightarrow$ TM

Proof.

By analysing the time complexity of the construction given to show that every non-deterministic TM N has an equivalent deterministic TM S.

For inputs of length n the computation tree of N has depth at most $t(n)$. Every tree node has at most b children, where $b \in \mathbb{N}$ depends on N's transition function. Thus the tree has no more than $b^{t(n)+1}$ nodes. S may have to explore all of them, in a breadth first fashion. Each exploration may take $O(t(n))$ steps (from the root to a node).

So all explorations together may take $O(t(n)) \cdot O(b^{t(n)+1}) = 2^{O(t(n))}$ time. $\square$

NTIME

Definition

NTIME$(t(n))$ is the class of languages decided by an $O(t(n))$ time non-deterministic TM.

Theorem

$$\text{TIME}(t(n)) \subseteq \text{NTIME}(t(n)) \subseteq \text{TIME}(2^{O(t(n))})$$

Space Complexity Classes

Definition

Let $f : \mathbb{N} \longrightarrow \mathbb{N}$.

Define the *space complexity classes*, **SPACE**$(f(n))$ and **NSPACE**$(f(n))$ to be the collection of all languages that are decidable by an $O(f(n))$ space TM, resp., NTM.

Space Complexity Classes

Definition

Let $f : \mathbb{N} \longrightarrow \mathbb{N}$.

Define the *space complexity classes*, **SPACE**$(f(n))$ and **NSPACE**$(f(n))$ to be the collection of all languages that are decidable by an $O(f(n))$ space TM, resp., NTM.

Space Complexity Example

Example

The TMs M_1 and M_2 for deciding $A = \{ 0^i 1^i \mid i \in \mathbb{N} \}$ use $s(n) = n$ space.

$$A \in \mathbf{SPACE}(n)$$

Space Complexity Example

Example

The TMs M_1 and M_2 for deciding $A = \{ 0^i 1^i \mid i \in \mathbb{N} \}$ use $s(n) = n$ space.

$$A \in \mathbf{SPACE}(n)$$

Can we do better?

Space Complexity Example

Example

The TMs M_1 and M_2 for deciding $A = \{ 0^i 1^i \mid i \in \mathbb{N} \}$ use $s(n) = n$ space.

$$A \in \mathbf{SPACE}(n)$$

Can we do better?

Not with these definitions, but we will see how to define sub-linear space complexity (Lecture 14).

Outline

Space vs Time

Theorem

For $f(n) \geq n$:

$$\begin{aligned} \text{TIME}(f(n)) &\subseteq \text{NTIME}(f(n)) \\ &\subseteq \text{SPACE}(f(n)) \\ &\subseteq \text{NSPACE}(f(n)) \\ &\subseteq \text{TIME}(f(n)2^{O(f(n))}) \end{aligned}$$

Proof sketch.

A TM that uses $f(n)$ time cannot write to more than $f(n)$ tape cells, so $\textbf{TIME}(f(n)) \subseteq \textbf{SPACE}(f(n))$.
A TM that uses $f(n)$ space and halts will pass through no more than $f(n)2^{O(f(n))}$ configurations, all of which must be distinct. □

Separating Complexity Classes

We have introduced several complexity classes, and have shown results of the form $\mathcal{C} \subseteq \mathcal{C}'$, but have generally had to say "we don't know if this containment is strict."

In general, separation results seem to be hard to prove.

But we do know *some*.

Space Constructibility

To state a space hierarchy theorem, we need the following technicality:

Definition

A function $f : \mathbb{N} \longrightarrow \mathbb{N}$ with $f(n) \geq \log n$ is *space constructible* if there exists a TM M that runs in space $O(f(n))$ that on input 1^n computes a binary representation of $f(n)$.

Space Hierarchy Theorem

Theorem

Let $f : \mathbb{N} \longrightarrow \mathbb{N}$ *be space constructible. Then there exists a language* A *that is decidable in space* $O(f(n))$ *but not in space* $o(f(n))$.

Corollary

if $f_1, f_2 : \mathbb{N} \longrightarrow \mathbb{N}$ *with* $f_1 = o(f_2)$ *and* f_2 *is space constructible, then* $\mathrm{SPACE}(f_1) \subsetneq \mathrm{SPACE}(f_2)$

Consequences of the Space-Hierarchy Theorem

Examples

- For $0 \leq \epsilon_1 < \epsilon_2$, $\mathbf{SPACE}(n^{\epsilon_1}) \subsetneq \mathbf{SPACE}(n^{\epsilon_2})$
- $\mathbf{SPACE}((\log n)^2) \subsetneq \mathbf{SPACE}(n)$
  - $\mathbf{NL} \subsetneq \mathbf{PSPACE}$
- $\mathbf{SPACE}(n^{\log n}) \subsetneq \mathbf{SPACE}(2^n)$
  - $\mathbf{PSPACE} \subsetneq \mathbf{EXPSPACE}$

Proof of Space Hierarchy Theorem I

Intuition: by Diagonalization. We construct a language $A \in \mathbf{SPACE}(f(n))$ that differs from the language accepted by any $o(f(n))$ space machine M on at least one string w_M.

In particular, we would like to pick $w_M = \langle M \rangle$ and have $\langle M \rangle \notin A$ iff M accepts $\langle M \rangle$.

Problem:

I. Simulating an arbitrary M that runs in space g within a fixed TM D requires space $c_M \cdot g(n)$ for some constant c_M that depends on M, since M may have more tape symbols than D.

II. $g = o(f)$ implies that $g(n) < \frac{1}{c_M} \cdot f(n)$, which resolves (1), but only for $n \geq n_0$, whereas we may have $|\langle M \rangle| < n_0$.

Solution: Simulate M for *multiple* inputs of the form $w = \langle M \rangle \# 1^k$.

Proof of Space Hierarchy Theorem II

Let A be the language decided by the following machine D that uses space $O(f(n))$:

On input w with $|w| = n$:

1. If w is of the form $\langle M \rangle \# 1^k$ for some TM M, continue, else *reject*
2. Compute $f(n)$ and mark out space $f(n)$. If later steps leave this area, *reject*
3. Simulate M on input w while counting computation steps:
   - if the count exceeds $2^{f(n)}$, reject
   - if M accepts, reject
   - if M rejects, accept

Proof of Space Hierarchy Theorem III

Claim

$A \notin \mathrm{SPACE}(o(f(n))$

NB

It is undecidable *whether a machine runs in space* $o(f(n))$*. But A doesn't care about machines that take more than this, we just need to make sure we have covered at least the space* $o(f(n))$ *ones.)*

Proof of Space Hierarchy Theorem III

Claim

$A \notin \text{SPACE}(o(f(n)))$

Proof.

Suppose M runs in space $g = o(f)$. We show by contradiction that M does not decide A. Suppose it does.

Let c_M be the tape symbol simulation cost factor for D to simulate M.

For some n_0 we have $n \geq n_0$ implies $c_M \cdot g(n) < f(n)$, so on input $w = \langle M \rangle \# 1^{n_0}$, the simulation of M on w runs in space $f(n)$ and terminates.

But then the decision of $D(w)$ and $M(w)$ are the opposite. So M does not decide $A = L(D)$. Contradiction. □

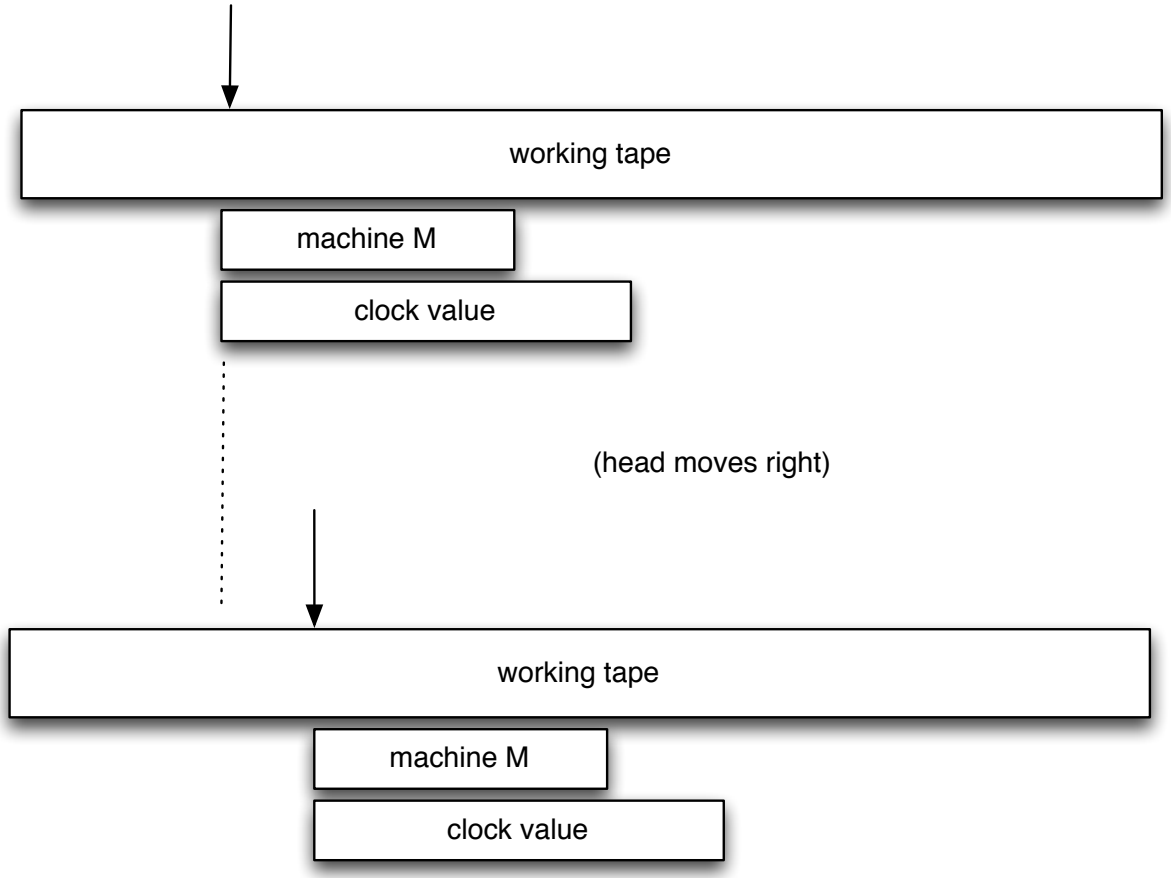
Time Hierarchy Theorem?

Can we get a similar separation result for time complexity classes?

Complication: simulating *one-tape* TM's for a time-bound t requires moving the head backwards and forwards between representations of the working tape, machine being simulated, and clock bits. This costs time!

Key idea: encode three tracks of information in each tape symbol of the simulating machine: working tape symbol, machine, clock. At each simulation step, move the machine and clock representations so that they stay near the position of the head on the working tape.

Time Hierarchy Theorem Construction



Now the overhead of simulation is a time factor of $O(|\langle M \rangle| + \log t)$.

Time Hierarchy Theorem

Definition

A function $t : \mathbb{N} \longrightarrow \mathbb{N}$ with $t(n) \geq n \log n$ is *time constructible* if the function that maps string 1^n to the binary representation of $t(n)$ is computable in time $O(t(n))$.

Theorem (Time Hierarchy Theorem)

For any time constructible function $t : \mathbb{N} \longrightarrow \mathbb{N}$ *there exists a language* A *that is decidable in time* $O(t(n))$ *but not decidable in time* $o(\frac{t(n)}{\log t(n)})$.

Proof.

Similar to proof of space hierachy theorem, but using the $|\langle M \rangle| + \log t$ simulation trick. The $|\langle M \rangle|$ is constant where it is needed in the proof. (See Sipser Thm 9.10) □

Consequences of the Time Hierarchy Theorem

Corollary

If $t_1, t_2 : \mathbb{N} \longrightarrow \mathbb{N}$ *satisfy* $t_1(n) = o(t_2(n)/\log t_2(n))$ *and* t_2 *is time constructible, then* $\mathrm{TIME}(t_1(n)) \subsetneq \mathrm{TIME}(t_2(n))$.

Examples

- For real numbers $1 \leq \epsilon_1 < \epsilon_2$, we have $\mathbf{TIME}(n^{\epsilon_1}) \subsetneq \mathbf{TIME}(n^{\epsilon_2})$.
- For all $k, \epsilon > 0$ we have $\mathbf{TIME}(n^k) \subsetneq \mathbf{TIME}((1+\epsilon)^n)$.
  - $\mathbf{P} \subsetneq \mathbf{EXPTIME}$